Link to Repository: https://github.com/yeswanth-123/FlexQL

FlexQL Design Document

1.Data Storage Architecture:

Decision: Hybrid PAX Storage with Optimized Memory & Disk Handling

- Data is stored using a Hybrid PAX model (page-wise + column-wise inside page).
- Each page contains 1,024 rows for better cache and memory alignment.
- Columns are separated inside pages (numeric as arrays, text as vectors).
- All numeric values stored as double to simplify processing.
- System uses pre-allocation and binary on-disk format (FQP2) for faster inserts and storage.

Justification:

This design is chosen to balance performance and simplicity. PAX reduces unnecessary data loading compared to row-store and avoids costly row reconstruction of column-store. Fixed page size improves cache efficiency, while pre-allocation speeds up inserts. The binary format further reduces storage size and avoids parsing overhead, making queries faster.

2.Indexing:

Decision:B+ Tree Indexing with Optimized RID & Insert Handling

- Uses an in-memory B+ Tree (order = 512) for indexing.
- Supports both exact lookup and range queries efficiently.
- Stores row location using RID encoding (page_id + slot).
- Provides multiple insert methods (single, bulk, bulk load) for efficiency.
- Index is rebuilt from disk instead of storing tree structure.

Justification:

B+ tree is chosen because it supports both fast search and range queries, unlike hash indexing. High order reduces tree depth, making lookups faster. RID encoding allows direct access to rows without extra mapping. Different insert methods reduce locking overhead, and rebuilding the tree from a flat file simplifies implementation while keeping good performance.

3.Caching:

Decision:LRU-Based Page Caching with Concurrency Optimization

- Uses a per-table LRU cache with capacity of 2,048 pages.
- Implements cache using hash map + deque for O(1) access and updates.
- Sequential scans bypass the cache to avoid unnecessary pollution.
- Uses 4,096 stripe locks for page-level concurrency control.
- Optimized for multi-threaded access with minimal locking overhead.

Justification:

LRU is chosen because it is simple and efficient, with low overhead compared to LFU or more complex policies. Cache bypass prevents large scans from removing useful data. Stripe locking allows multiple threads to access pages safely without heavy contention, improving overall performance.

4.Expiration Timestamps:

Decision: Lazy Expiration with Tombstone Fusion

- Each row stores an expiration timestamp (EXPIRES_AT).
- Expiry is checked only at query time (lazy expiry).
- Default expiration is set if not provided during insert.
- Combines expiry check and deletion (tombstone) check in one condition.
- No background thread is used for cleaning expired data.

Justification:

Lazy expiry is chosen to avoid the overhead of background threads and complex locking. It ensures expired rows are ignored immediately during queries with minimal cost. Combining expiry and tombstone checks reduces CPU overhead and improves performance.

5.Multithreading:

Decision: Thread Pool with Multi-Level Locking and Parallel Execution

- Uses a fixed thread pool instead of creating threads per connection.
- Implements three-layer locking (global, table-level, page-level).
- Supports parallel full-table scans for large datasets.
- Uses shared and exclusive locks for safe concurrent access.
- Separates file write locks (data and WAL) for better performance.

Justification:

Thread pool reduces overhead of creating threads repeatedly and improves efficiency. Multi-level locking balances safety and performance by reducing contention. Parallel scans speed up large queries, while separate locks for files allow concurrent writes without unnecessary blocking.

- The system supports commands like CREATE, USE, and DROP DATABASE, along with basic SQL operations.

Other Design Choices:

1.Write-Ahead Log:

Decision: Write-Ahead Logging (WAL) for Crash Recovery:

- Uses WAL (append-only logging) before writing to main data file.
- Batches multiple inserts into single write operations (writev).
- Uses a simple log format with row tracking for recovery.
- Performs periodic checkpointing to limit log size.
- On restart, replays WAL entries based on row count.

Justification:

WAL is chosen to prevent data loss during crashes while maintaining high performance. It avoids expensive disk operations like fsync per row and reduces write overhead compared to shadow paging. The design is simple, efficient, and ensures reliable recovery with minimal runtime cost.

2.Network Protocol:

Decision: Efficient Length-Prefixed Network Protocol

- Uses length-prefixed messages for sending queries and results.
- Uses special separators (\x1E, \x1F) for rows and columns.
- Sends results using a single buffered write.
- Enables TCP_NODELAY to reduce network latency.
- Supports batch INSERT (up to 5,000 rows) in one request.

Justification:

This design ensures reliable and fast communication between client and server. Length-prefix avoids parsing issues, while special separators prevent conflicts with user data. Single write and TCP_NODELAY reduce network delays, and batch inserts improve throughput by reducing communication overhead.

3.Query Parser Design:

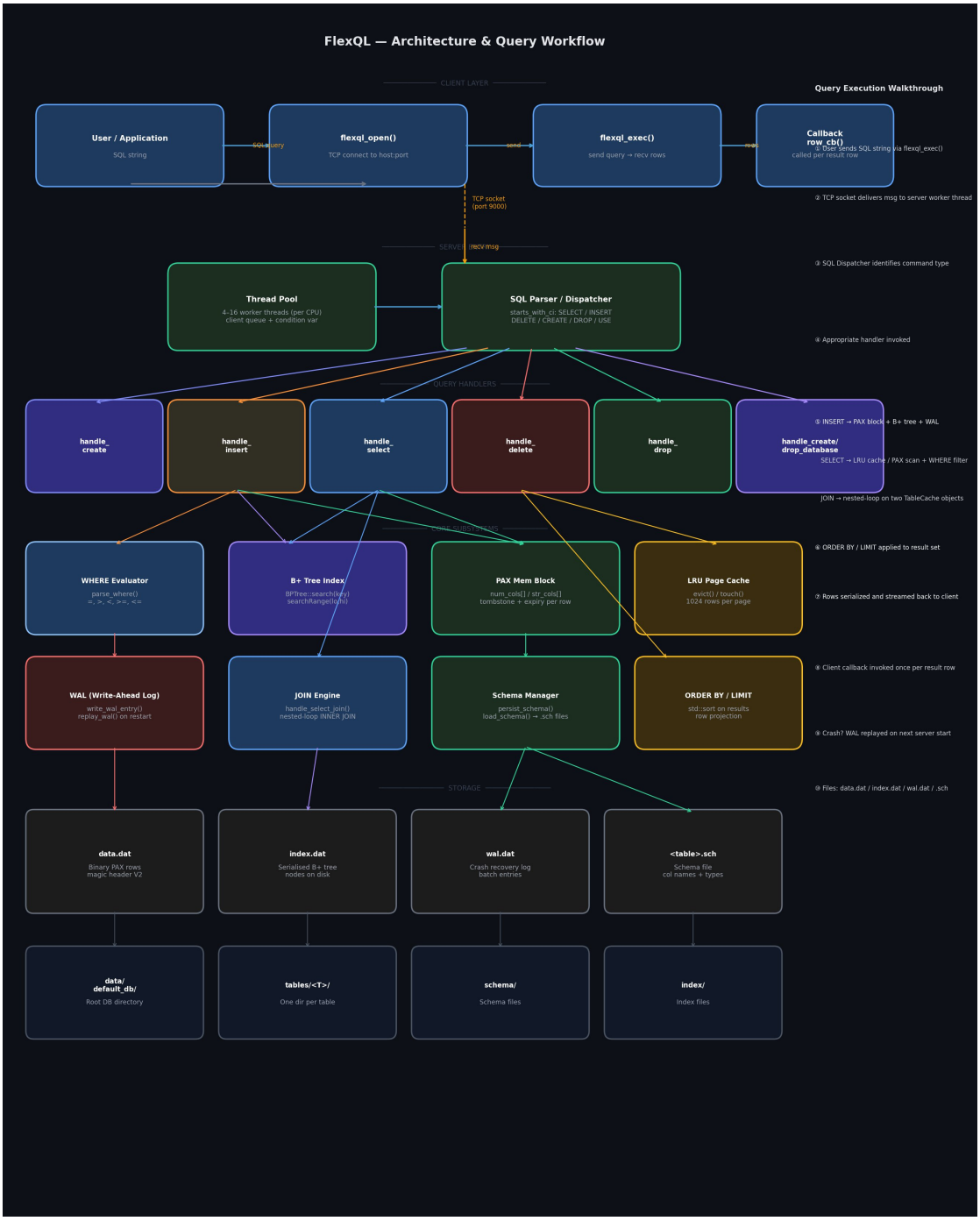
Decision: Lightweight Hand-Written Query Parser

- Uses a hand-written recursive descent parser (no external tools).
- Supports case-insensitive SQL keywords using simple conversion.
- Uses std::string_view to avoid unnecessary string copying.
- Implements a custom fast number parser for numeric values.
- Designed for single-pass parsing with minimal overhead.

Justification:

This design keeps the parser simple, fast, and easy to debug. Using string_view avoids heavy memory allocations during large inserts, and the custom number parser improves performance. Since the SQL subset is small, a hand-written parser is more efficient than using complex external tools.

FlexQL Architecture AND WorkFlow:



FlexQL — Workflow Summary:

PHASE 1 — SERVER STARTUP:

Server calls load_all_databases() to read all .sch schema files from disk and rebuild every TableCache in memory. Then it opens a TCP socket on port 9000 and spawns 4–16 worker threads (based on CPU count) that wait on a condition variable for client work.

PHASE 2 — CLIENT CONNECTS:

flexql_open() establishes a TCP connection. The server's accept loop pushes the socket into a shared queue. A worker thread picks it up and enters handle_client(), which loops for the lifetime of that connection.

PHASE 3 — QUERY RECEIVED:

flexql_exec() length-prefixes the SQL and sends it over the socket. The worker reads the 4-byte header, then the full query, trims it, and does a fast case-insensitive prefix check to classify the statement type.

PHASE 4 — DISPATCH:

INSERT INTO → handle_insert

CREATE TABLE → handle_create

SELECT → handle_select (branches on INNER JOIN)

DELETE FROM → handle_delete

DROP TABLE → handle_drop_table

USE / CREATE DATABASE / DROP DATABASE → respective handlers

PHASE 5a — INSERT:

Write lock acquired. VALUES clause parsed with zero-copy string_view — no heap alloc per row. Numeric values appended to num_cols[], strings to str_cols[] (PAX layout). A tombstone byte + expiry timestamp added per row. Index key/RID pairs bulk-loadedinto the B+ tree. WAL entry written to wal.dat before confirming. Every 200 batchesWAL is checkpointed (truncated). Reply: OK.

PHASE 5b — SELECT (no JOIN):

Read lock acquired. If WHERE targets the index column, BPTree::search() or searchRange() fetches only matching row IDs — no full scan. Otherwise, pages of 1024 rows are checked in the LRU cache first; on miss, live rows are read from the PAX block and cached. WHERE filter applied per row. ORDER BY sorts results with std::sort, LIMIT truncates. Rows sent back as pipe-delimited length-prefixed messages.

PHASE 5c — SELECT (INNER JOIN):

Read locks on both tables. Nested-loop join: each row from the left table that passes WHERE is matched against the right table on the ON column. Matched rows projected and sent back.

PHASE 5d — DELETE:

Write lock acquired. Matching rows have their tombstone byte set to 1 in the PAX block. data.dat is then rewritten from scratch skipping tombstoned rows, and index.dat is rebuilt clean.

PHASE 5e — CREATE TABLE:

Column list parsed, types mapped (INT→0, DECIMAL→1, VARCHAR→2). New TableCache allocated, PAX arrays initialised, table directory created, magic headers written to data.dat and index.dat, schema saved to schema/<table>.sch. TableCache registered in the global databases map.

PHASE 6 — RESPONSE TO CLIENT:

Each result row is sent as a 4-byte length + pipe-delimited string. flexql_exec reads these, splits on |, and fires the user's callback once per row. Final OK or ERROR:... message ends the call and sets the return code.

PHASE 7 — CRASH RECOVERY:

On next startup, replay_wal() checks how many rows data.dat already has and re-applies only the missing WAL entries. WAL is then reset to its magic header. Database is fully consistent before any client is accepted.

STORAGE LAYOUT:

```
data/
  default_db/
    tables/USERS/
      data.dat    —  binary PAX rows (page-chunked, magic header V2)
      index.dat   —  serialised B+ tree nodes
      wal.dat     —  write-ahead log
    schema/
      USERS.sch   —  INDEX_COL + col_name:type (plain text)
```